

Rapid software development

- Rapid software development processes are designed to produce useful **software Quickly**.
- These processes do not follow a strict, linear sequence (like traditional Waterfall models) but instead use an **iterative approach**.
- This means that different phases—**specification, design, development, and testing**—are not completed one after another but are **interleaved** (overlapping and repeated in cycles).

Key Characteristics of Rapid Software Development

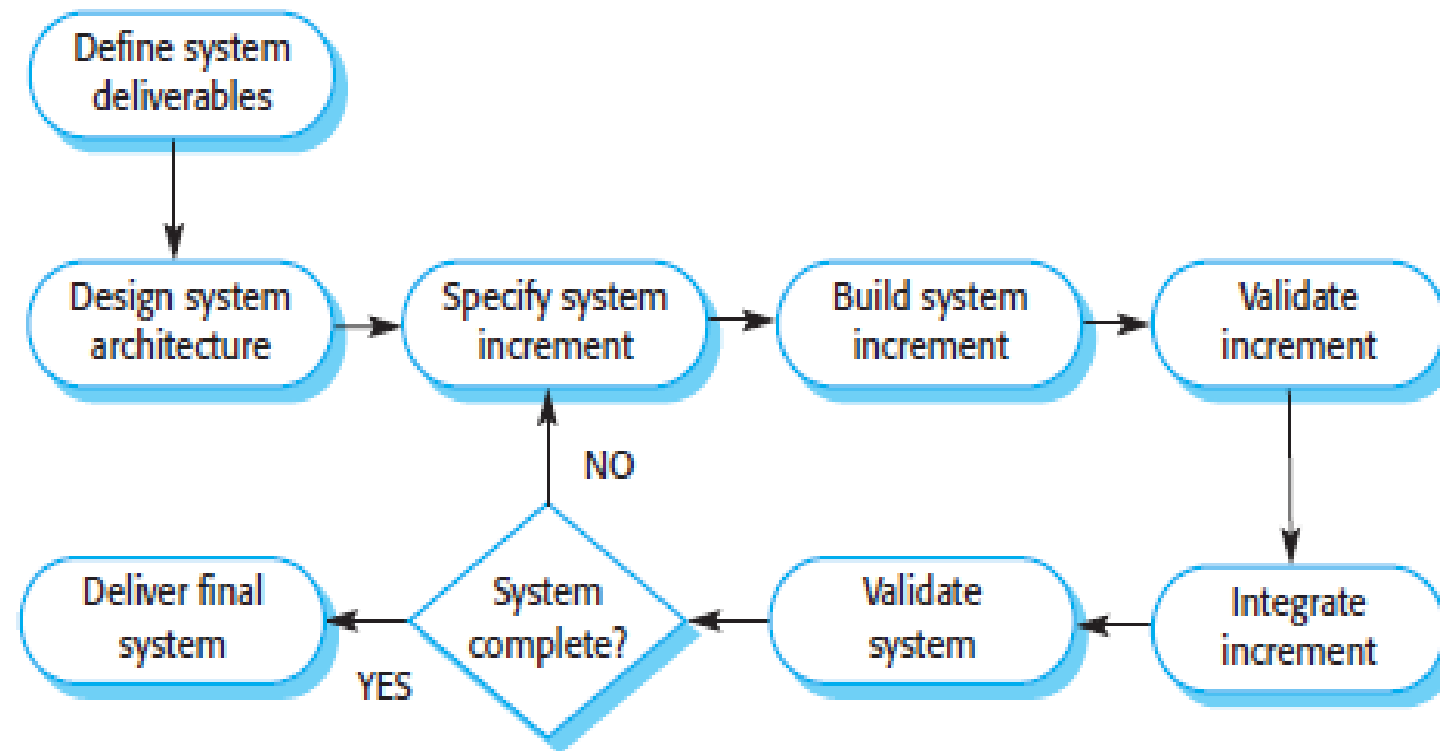
- **Concurrent Development:** Specification, design, and implementation happen simultaneously. Detailed documentation is minimized, and only essential requirements are defined.
- **Incremental Delivery:** The system is built in stages, with user feedback shaping each iteration. New requirements can be added in later increments.

Key Characteristics of Rapid Software Development

- **Interactive UI Development:** User interfaces are created quickly using drag-and-drop tools, generating web-based or platform-specific interfaces automatically.

An Iterative development Process

Figure 17.1
An iterative
development process



Advantages of incremental development

- Accelerated delivery of customer services. Each increment delivers the highest priority functionality to the customer.
- User engagement with the system. Users have to be involved in the development which means the system is more likely to meet their requirements and the users are more committed to the system.

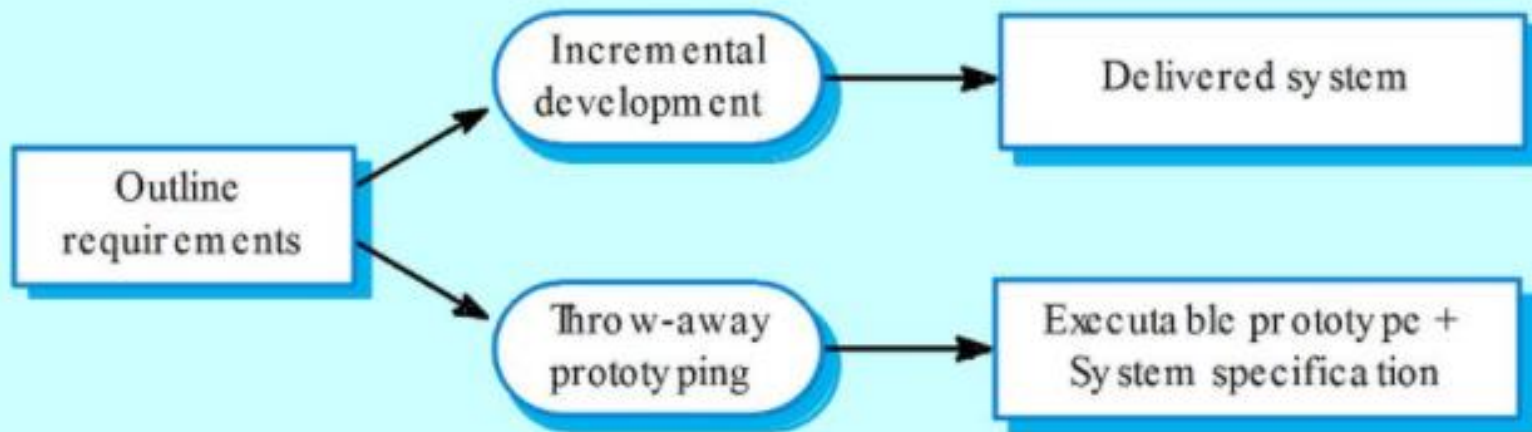
Problems with incremental development

- Management problems
 - Progress can be hard to judge and problems hard to find because there is no documentation to demonstrate what has been done.
- Contractual problems
 - The normal contract may include a specification; without a specification, different forms of contract have to be used.
- Validation problems
 - Without a specification, what is the system being tested against?
- Maintenance problems
 - Continual change tends to corrupt software structure making it more expensive to change and evolve to meet new requirements.

Prototyping

- For some large systems, incremental iterative development and delivery may be impractical; this is especially true when multiple teams are working on different sites.
- Prototyping, where an experimental system is developed as a basis for formulating the requirements may be used. This system is thrown away when the system specification has been agreed.

Incremental development and prototyping



Conflicting objectives

- The objective of **incremental development** is to deliver a working system to end-users. The development starts with those requirements which are best understood.
- The objective of **throw-away prototyping** is to validate or derive the system requirements. The prototyping process starts with those requirements which are poorly understood.

1. Agile methods

- Dissatisfaction with the overheads involved in design methods led to the creation of Agile methods. These methods:
 - Focus on the code rather than the design;
 - Are based on an iterative approach to software development;
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- Agile methods are probably best suited to small/medium-sized business systems or PC products.

Principles Of Agile Methods

Figure 17.3 The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognised and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change, so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Problems with Agile methods

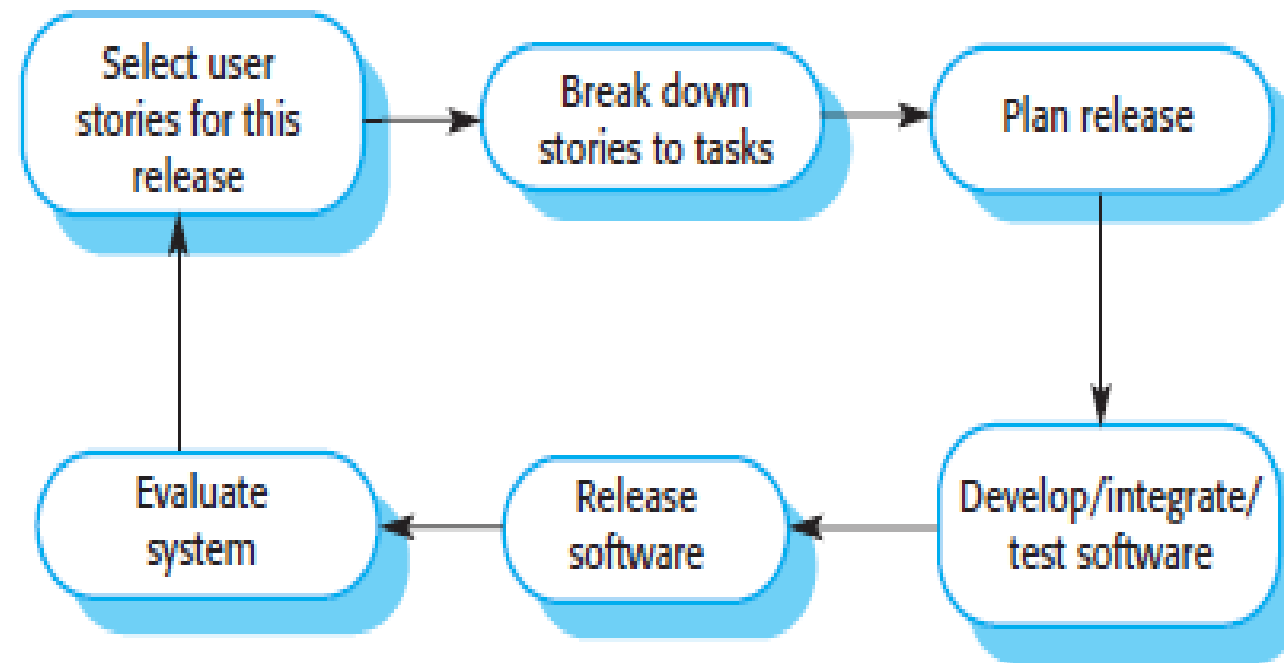
- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterizes agile methods.
- Prioritizing changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work.
- Contracts may be a problem as with other approaches to iterative development.

2. Extreme programming

- Perhaps the best-known and most widely used agile method.
- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
 - New versions may be built several times per day;
 - Increments are delivered to customers every 2 weeks;
 - All tests must be run for every build and the build is only accepted if tests run successfully.

Extreme programming release Cycle

Figure 17.4 The extreme programming release cycle



Extreme programming practices 1

Incremental planning	Requirements are recorded on Story Cards and the Stories to be included in a release are determined by the time available and their relative priority. The developers break these Stories into development Tasks
Small Releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple Design	Enough design is carried out to meet the current requirements and no more.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme programming practices 2

Pair Programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective Ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers own all the code. Anyone can change anything.
Continuous Integration	As soon as work on a task is complete it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of over-time are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site Customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

XP and Agile Principles

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant re-factoring of code.

Requirements scenarios

- In XP, user requirements are expressed as scenarios or user stories.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

XP and change

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes constant code improvement (re-factoring) to make changes easier when they have to be implemented.

Testing in XP

- Test-first development.
- Incremental test development from scenarios.
- User involvement in test development and validation.
- Automated test harnesses are used to run all component tests each time that a new release is built.

Test-first development

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
- All previous and new tests are automatically run when new functionality is added. Thus checking that the new functionality has not introduced errors.

Figure 17.8 Test case description for credit card validity

Test 4: Test credit card validity

Input:

A string representing the credit card number and two integers representing the month and year when the card expires

Tests:

Check that all bytes in the string are digits

Check that the month lies between 1 and 12 and the year is greater than or equal to the current year.

Using the first 4 digits of the credit card number, check that the card issuer is valid by looking up the card issuer table. Check credit card validity by submitting the card number and expire date information to the card issuer

Output:

OK or error message indicating that the card is invalid

Pair programming

- In XP, programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this.
- Measurements suggest that development productivity with pair programming is similar to that of two people working independently.

Rapid Application Development (RAD)

RAD is an iterative approach primarily used for **data-intensive applications**, evolving from **fourth-generation languages** in the 1980s.

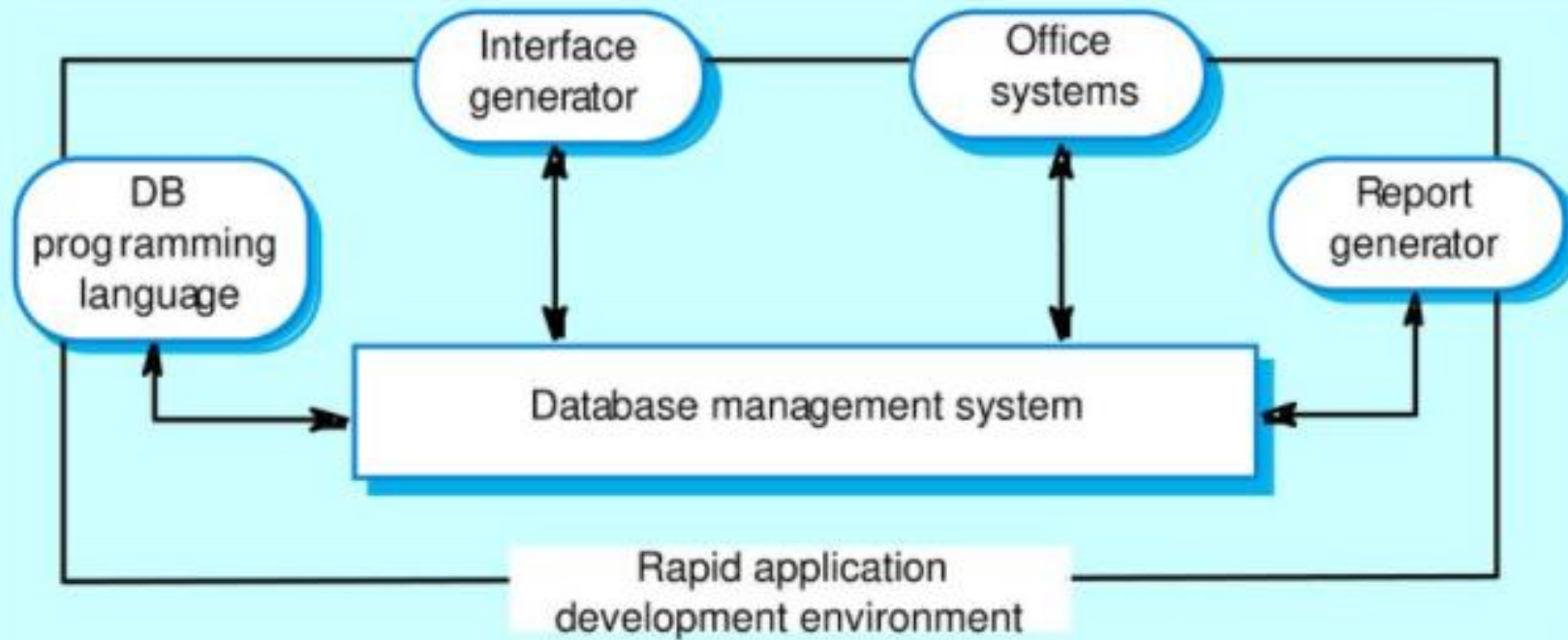
Key RAD Tools:

- 1.Database Programming Language:** Embeds database operations (e.g., SQL) for data manipulation.
- 2.Interface Generator:** Creates user-friendly forms for data input and display.

Rapid Application Development (RAD)

- 1.Office Application Links:** Connects to spreadsheets and word processors for data analysis and reporting.
- 2.Report Generator:** Defines and creates reports from database information.

A RAD environment



Interface generation

- Many applications are based around complex forms and developing these forms manually is a time-consuming activity.
- RAD environments include support for screen generation including:
 - Interactive form definition using drag and drop techniques;
 - Form linking where the sequence of forms to be presented is specified;
 - Form verification where allowed ranges in form fields is defined.

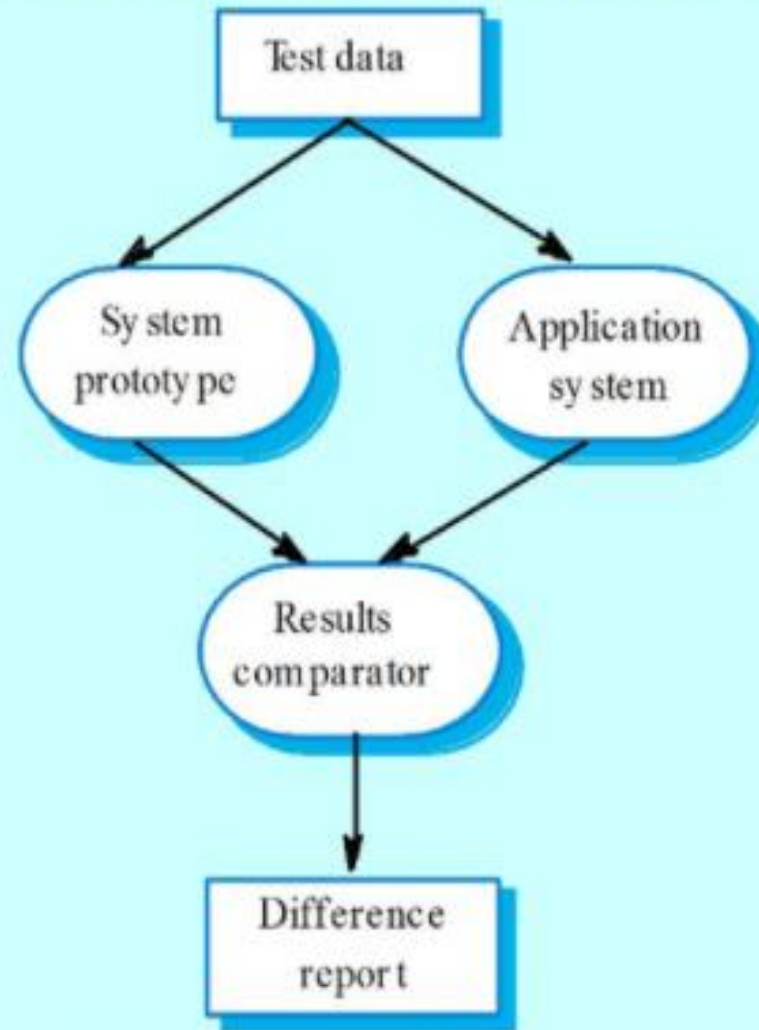
4. Software prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- A prototype can be used in:
 - The requirements engineering process to help with requirements elicitation and validation;
 - In design processes to explore options and develop a UI design;
 - In the testing process to run back-to-back tests.

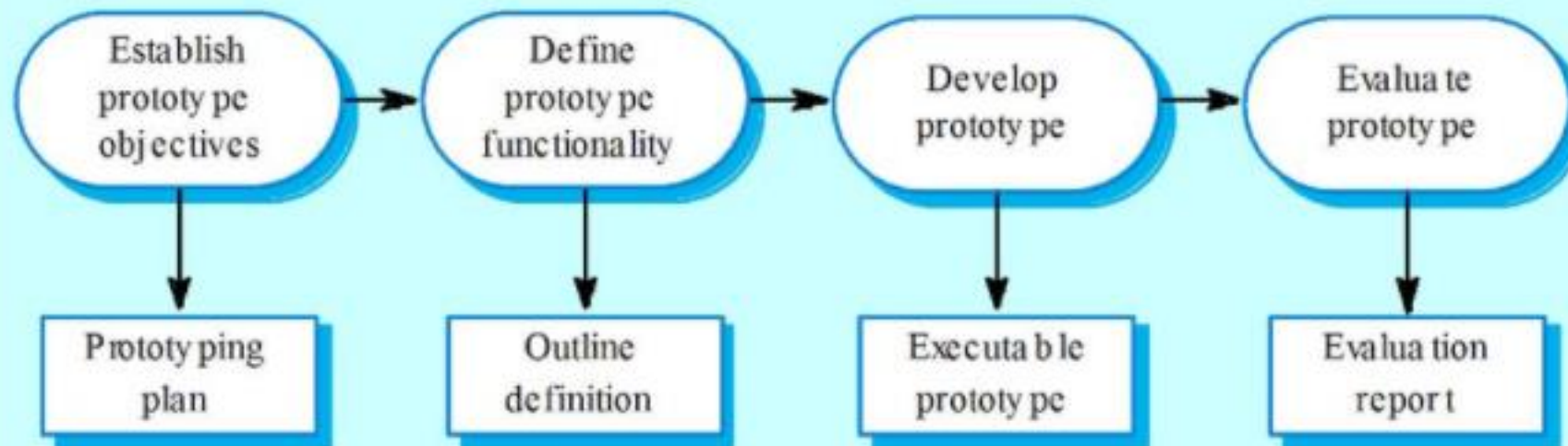
Benefits of prototyping

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

Back to back testing



The prototyping process



Throw-away prototypes

- Prototypes should be discarded after development as they are not a good basis for a production system:
 - It may be impossible to tune the system to meet non-functional requirements;
 - Prototypes are normally undocumented;
 - The prototype structure is usually degraded through rapid change;
 - The prototype probably will not meet normal organizational quality standards.

Key Differences & When to Use Each Approach

Method	Key Focus	Best Used For
Agile	Iterative development, collaboration, adaptability	Large, evolving projects (e.g., web apps, enterprise software)
XP	Code quality, continuous testing, frequent releases	High-risk projects (e.g., fintech, healthcare software)
RAD	Speed, quick feedback, minimal planning	Short-term projects (e.g. internal tools)
Prototyping	Visualizing concepts, early user feedback	UI/UX-heavy projects (e.g., mobile apps, web platforms)